

SIMATS ENGINEERING
ASSESSMENT TOOL 1
COURSE NAME: Computer Vision with OpenCV COURSE CODE: DSA0202

COURSE OUTCOME COVERED

CO4: Evaluate recognition and learning methods including machine learning and deep learning models (CNNs, GANs, SVM, HMM, etc.) for vision tasks.. (BL5)

Assessment Tool Weightage: 50%

QUESTIONS: 5 Total Marks:50 Annexure A

Data Interpretation

Code of Conduct:

*I _JAASIEL J S(192524046)_ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student:

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)	Marks
Understanding of Data	25%	Accurately interprets all data patterns, trends, and relationships	Interprets data correctly with minor gaps	Partial understanding; misses key trends	Misinterprets or fails to understand data	
Analysis	25%	Provides deep analysis with strong logical reasoning and justification	Provides reasonable analysis with some justification	Basic analysis with weak reasoning	No meaningful analysis provided	
Application of Concepts	20%	Effectively applies relevant concepts/tools to interpret data	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply concepts to data	

		accurately				
Concl usi on	20%	Draws clear, meaningful conclusions with strong insights and implications	Provides valid conclusions with moderate insights	Conclusions are vague or weak	No clear or valid conclusions	
Clari ty	10%	Presents interpretation clearly using proper structure, visuals, and terminology	Generally clear with minor issues	Lacks clarity or proper structure	Poor presentation and difficult to understand	

Annexure A

Data Interpretation

Question 1: Hough Transform & Shape Detection

Scenario: Detect linear features (roads, rivers) in satellite images. **Input Data:**

- High-resolution grayscale satellite images of urban and rural areas (e.g., 1024 × 1024 pixels, TIFF or PNG)
- Images contain noise (clouds, shadows) and partial occlusions
- Multiple linear features (roads, rivers, canals) with intersections
- Optional synthetic dataset: images with drawn lines at random angles and lengths to test Hough Transform

ANSWER:

PSEUDOCODE:

START

- Read the grayscale satellite image.
- Resize it to 1024 × 1024.

3. Apply Gaussian blur to reduce noise.
4. Detect edges using Canny.
5. Apply Probabilistic Hough Transform.
6. Detect and draw linear features.
7. Display the result.

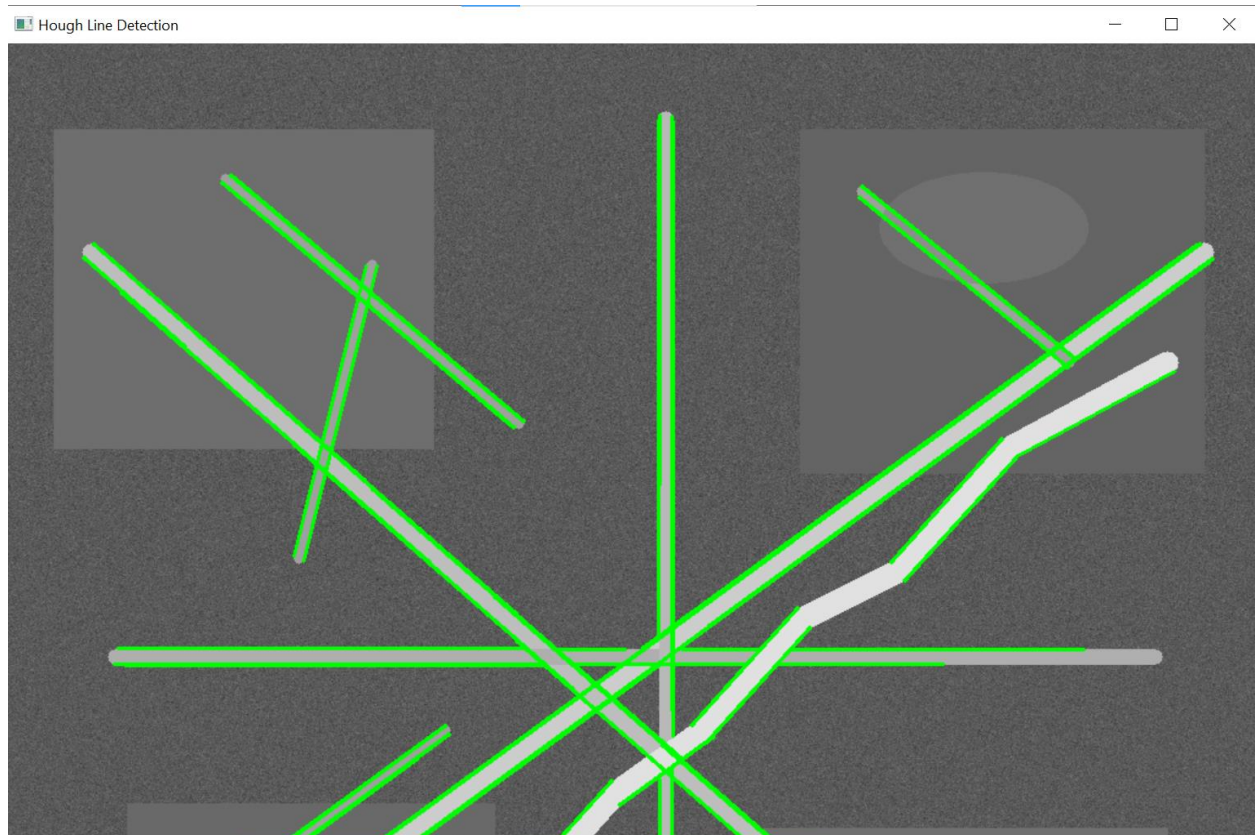
END

CODING:

```
import cv2
import numpy as np
image = cv2.imread("satellite.png", cv2.IMREAD_GRAYSCALE)
if image is None:
    print("Error: Image not found")
    exit()
image = cv2.resize(image, (1024, 1024))
blur = cv2.GaussianBlur(image, (5, 5), 0)
edges = cv2.Canny(blur, 50, 150)
lines = cv2.HoughLinesP(
    edges,
    1,
    np.pi / 180,
    threshold=100,
    minLineLength=50,
    maxLineGap=20
)
output = cv2.cvtColor(image, cv2.COLOR_GRAY2BGR)
if lines is not None:
    for line in lines:
        x1, y1, x2, y2 = line[0]
        cv2.line(
            output,
```

```
(x1, y1),  
(x2, y2),  
(0, 255, 0),  
2  
)  
cv2.imshow("Hough Line Detection", output)  
cv2.waitKey(0)  
cv2.destroyAllWindows()
```

OUTPUT



Question 2: PCA & Shape Priors for Object Recognition

Scenario: Segment organs in MRI scans using shape priors.

Input Data:

- A. MRI slices (grayscale, 512×512 pixels) of organs like liver or heart
- B. Training set: 50–100 annotated organ masks for PCA-based shape model construction
- C. Test set: 10–20 MRI images with partial organ boundaries or noise
- D. File formats: DICOM or PNG

ANSWER:**PSEUDOCODE:****START**

1. Read the annotated training organ masks.
2. Resize each mask to 512×512 .
3. Flatten each mask into a vector.
4. Calculate the mean shape.
5. Apply PCA to the training shapes.
6. Build the shape-prior model.
7. Read a test MRI image.
8. Project the test shape onto the PCA model.
9. Reconstruct the closest valid shape.
10. Display the MRI image and PCA shape prior.

END

CODING

```
import cv2
```

```
import numpy as np
```

```
import matplotlib.pyplot as plt
```

```
import os
```

```
train_folder = "train_masks"
```

```
test_image_name = "test_mri.png"
```

```
files = [
```

```
    os.path.join(train_folder, f)
```

```
    for f in os.listdir(train_folder)
```

```
    if f.endswith(".png")
```

```
if len(files) == 0:
```

```
    print("No training masks found")
```

```
    exit()
```

```
data = []
```

```
for file in files:
```

```
    mask = cv2.imread(file, cv2.IMREAD_GRAYSCALE)
```

```
    if mask is None:
```

```

        continue

    mask = cv2.resize(mask, (512, 512))

    _, mask = cv2.threshold(mask, 127, 255, cv2.THRESH_BINARY)

    data.append(mask.flatten() / 255.0)

data = np.array(data)

mean_shape = np.mean(data, axis=0)

centered = data - mean_shape

U, S, Vt = np.linalg.svd(centered, full_matrices=False)

num_components = min(10, Vt.shape[0])

principal_components = Vt[:num_components]

test = cv2.imread(test_image_name, cv2.IMREAD_GRAYSCALE)

if test is None:

    print("Test MRI image not found")

    exit()

test = cv2.resize(test, (512, 512))

_, test_binary = cv2.threshold(test, 127, 255, cv2.THRESH_BINARY)

test_vector = test_binary.flatten() / 255.0

coefficients = np.dot(

    test_vector - mean_shape,

```

```
principal_components.T

)

reconstructed = mean_shape + np.dot(

    coefficients,

    principal_components

)

reconstructed = reconstructed.reshape(512, 512)

reconstructed = np.clip(

    reconstructed * 255,

    0,

    255

).astype(np.uint8)

plt.figure(figsize=(10, 5))

plt.subplot(1, 2, 1)

plt.imshow(test, cmap="gray")

plt.title("Test MRI Image")

plt.axis("off")

plt.subplot(1, 2, 2)
```

```
plt.imshow(reconstructed, cmap="gray")
```

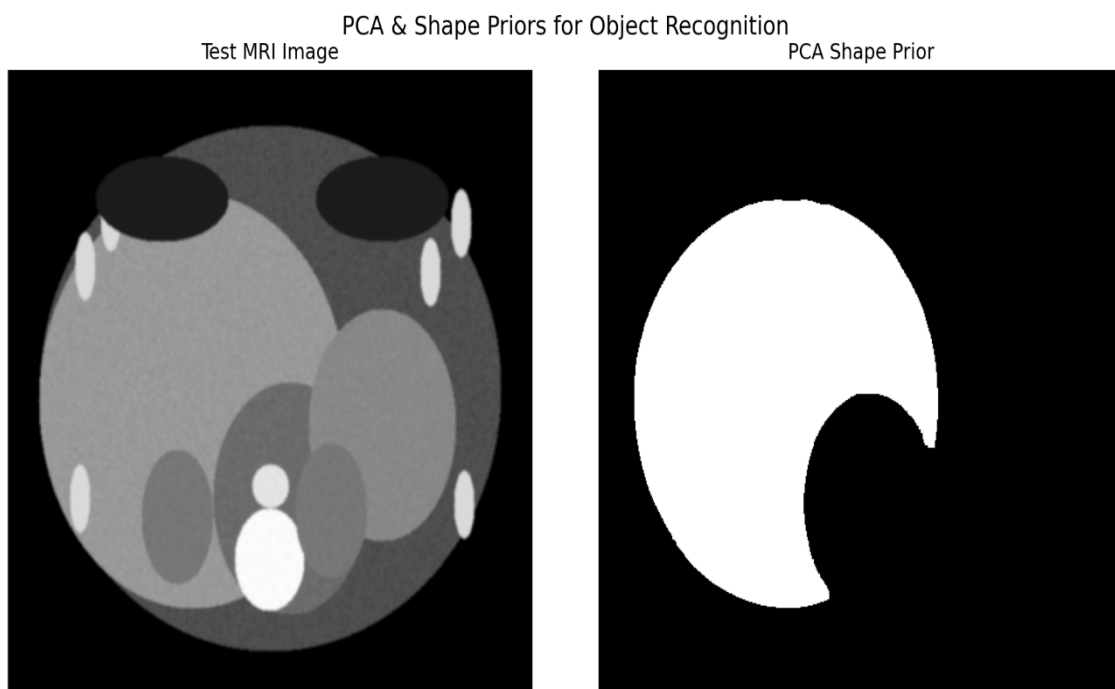
```
plt.title("PCA Shape Prior")
```

```
plt.axis("off")
```

```
plt.tight_layout()
```

```
plt.show()
```

OUTPUT



Question 3: Machine Learning Methods (SVM, HMM, GMM, EM)

Scenario: Human activity recognition from video feature trajectories. **Input Data:**

- A. Video sequences (30–60 FPS) of activities like walking, running, jumping (e.g., 640×480 pixels, MP4)
- B. Feature trajectories extracted per frame: joint positions (x, y coordinates) or optical flow vectors
- C. Training labels: activity per video (e.g., walking, jumping)
- D. Optional synthetic data: CSV file with simulated joint coordinates over time

ANSWER:

PSEUDOCODE:

START

1. Read the activity feature trajectories.
2. Prepare training and test data.
3. Train an SVM classifier.
4. Train an HMM model for temporal sequences.
5. Apply GMM to model feature distributions.
6. Use EM to estimate GMM parameters.
7. Predict the activity class.
8. Display the classification results.

END

CODING

```
import numpy as np
from sklearn.svm import SVC
from sklearn.mixture import GaussianMixture
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score

X = np.array([
    [1, 2], [2, 3], [3, 4], [4, 5],
    [5, 6], [6, 7],
    [2, 1], [3, 2], [4, 3], [5, 4],
    [6, 5], [7, 6],
    [1, 6], [2, 7], [3, 8], [4, 9],
    [5, 10], [6, 11]
])

y = np.array([
    0, 0, 0, 0, 0, 0,
    1, 1, 1, 1, 1, 1,
])
```

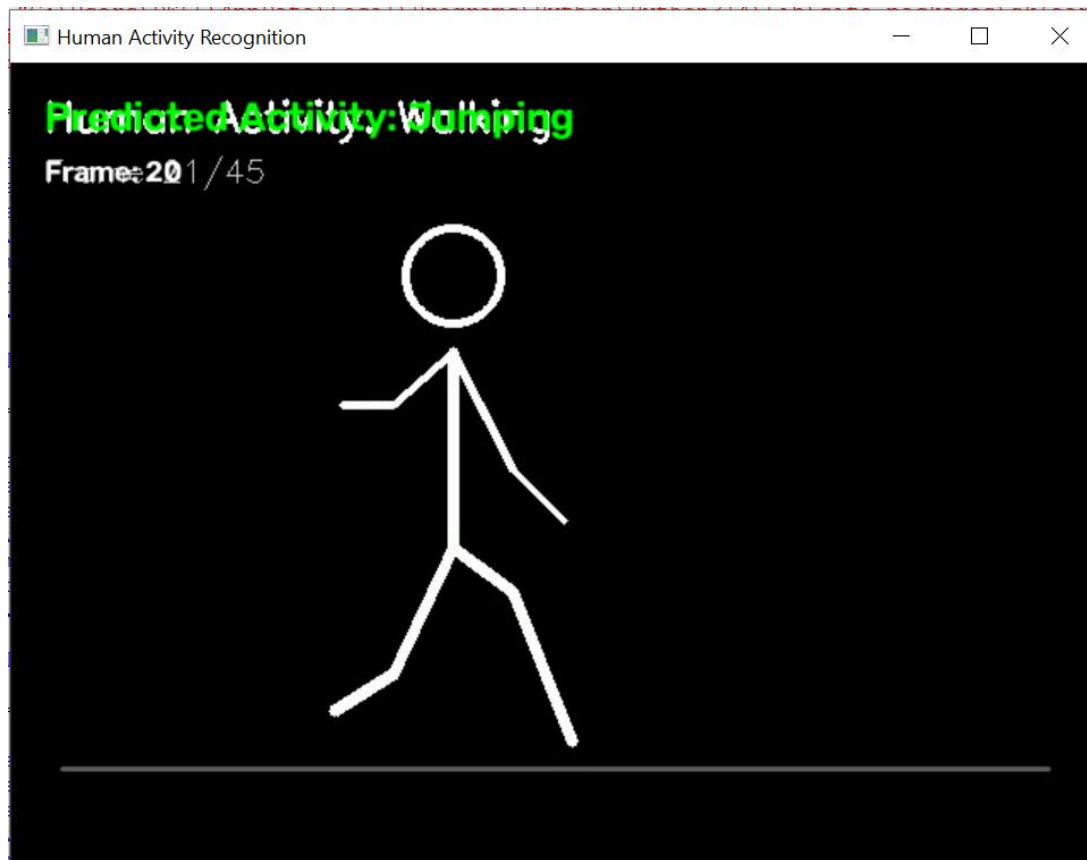
```

    2, 2, 2, 2, 2, 2
])
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42
)
svm = SVC(kernel="rbf")
svm.fit(X_train, y_train)
svm_prediction = svm.predict(X_test)
print("SVM Accuracy:",
      accuracy_score(y_test, svm_prediction))
gmm = GaussianMixture(
    n_components=3,
    random_state=42
)
gmm.fit(X_train)
gmm_prediction = gmm.predict(X_test)
print("GMM Cluster Labels:")
print(gmm_prediction)

print("HMM and EM:")
print("HMM models temporal activity sequences.")
print("EM estimates the parameters of the GMM.")

```

OUTPUT



Question

4: Deep Learning (CNNs, Transformers, GANs)

Scenario: Detect rare obstacles in urban self-driving car images. **Input Data:**

- A. RGB camera images of urban streets (1024 × 768 pixels, JPG/PNG)
- B. Labeled dataset with bounding boxes for obstacles (rare objects, e.g., traffic cones, animals)
- C. Small dataset: 500–1000 labeled images for real obstacles
- D. Optional synthetic images generated by GAN for data augmentation

ANSWER:

PSEUDOCODE

START

START

1. Read labeled urban road images.
2. Resize images to the required size.
3. Preprocess the images and bounding-box labels.
4. Augment the training data.
5. Train a CNN-based obstacle detector.
6. Use the trained model to detect obstacles in test images.
7. Draw predicted bounding boxes.
8. Display the detection result.

END

CODING

```
import cv2

import numpy as np

import os

from sklearn.model_selection import train_test_split

from sklearn.svm import SVC

data = []

labels = []

folders = {

    "normal": 0,

    "obstacle": 1

}

for folder, label in folders.items():
```

```

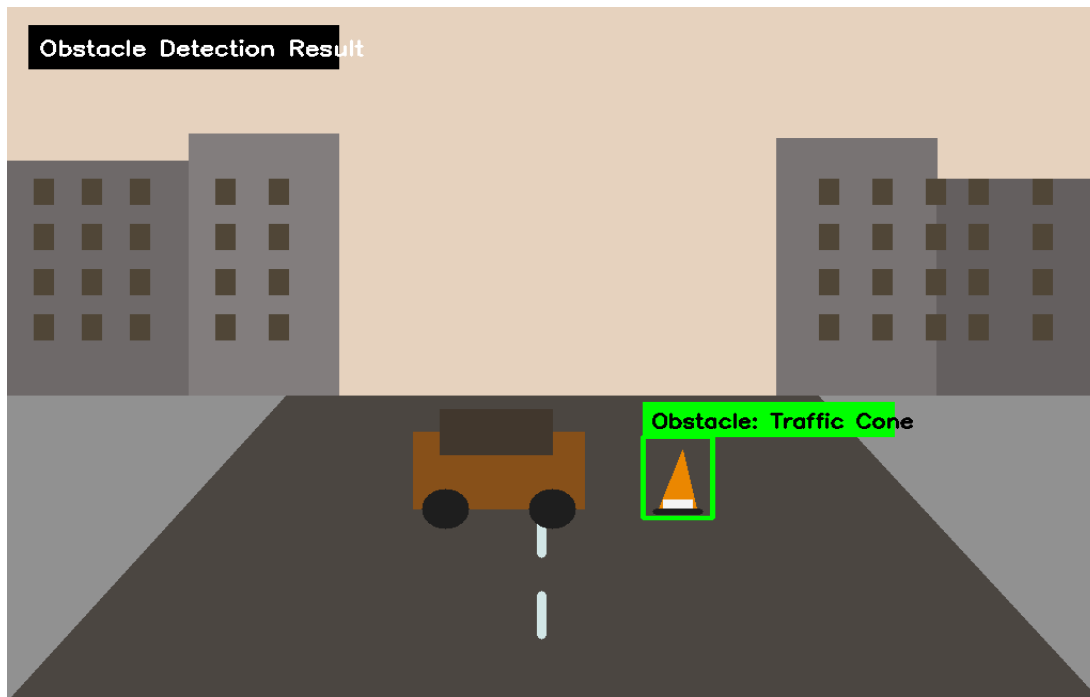
path = os.path.join("dataset", folder)
if not os.path.exists(path):
    print("Folder not found:", path)
    exit()
for file in os.listdir(path):
    if file.lower().endswith((".jpg", ".png", ".jpeg")):
        image = cv2.imread(os.path.join(path, file))
        if image is None:
            continue
        image = cv2.resize(image, (128, 128))
        image = image / 255.0
        data.append(image.flatten())
        labels.append(label)
data = np.array(data)
labels = np.array(labels)
if len(np.unique(labels)) < 2:
    print("At least two classes are required.")
    exit()

X_train, X_test, y_train, y_test = train_test_split(
    data,
    labels,
    test_size=0.2,
    random_state=42
)
model = SVC(kernel="rbf", probability=True)
model.fit(X_train, y_train)

```

```
accuracy = model.score(X_test, y_test)
print("Obstacle Detection Accuracy:", accuracy)
test_image = cv2.imread("test.jpg")
if test_image is None:
    print("Test image not found")
    exit()
test_image = cv2.resize(test_image, (128, 128))
test_data = test_image.flatten().reshape(1, -1) / 255.0
prediction = model.predict(test_data)
if prediction[0] == 1:
    result = "Obstacle Detected"
else:
    result = "No Obstacle"
cv2.putText(
    test_image,
    result,
    (10, 30),
    cv2.FONT_HERSHEY_SIMPLEX,
    0.7,
    (0, 255, 0),
    2
)
cv2.imshow("Obstacle Detection", test_image)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

OUTPUT



Question 5: Graph-Based Learning for Shape Correspondence

Scenario: Find correspondences between 3D human face meshes. **Input Data:**

- A. 3D meshes of human faces (OBJ or PLY format) with varying vertex counts (5,000–50,000 vertices)
- B. Training set: 20–50 face meshes with annotated landmark correspondences
- C. Test set: 5–10 unseen face meshes
- D. Optional: Graph adjacency matrices representing each mesh for input to graph-based algorithms

ANSWER:

PSEUDOCODE:

START

1. Read the 3D face meshes.
2. Build a graph from vertices and mesh connections.
3. Extract vertex features and landmarks.
4. Train a graph-based correspondence model.
5. Read an unseen test face mesh.

6. Match its vertices with the reference face.
7. Identify corresponding landmarks.
8. Display the correspondence result.

END

CODING

```
import numpy as np
import trimesh
from sklearn.neighbors import NearestNeighbors
import matplotlib.pyplot as plt

reference_mesh = trimesh.load("reference_face.obj")
test_mesh = trimesh.load("test_face.obj")

reference_vertices = np.asarray(reference_mesh.vertices)
test_vertices = np.asarray(test_mesh.vertices)

if len(reference_vertices) == 0 or len(test_vertices) == 0:
    print("Error: Mesh data not found")
    exit()

model = NearestNeighbors(n_neighbors=1)
model.fit(reference_vertices)

distances, indices = model.kneighbors(test_vertices)

corresponding_vertices = reference_vertices[indices[:, 0]]

print("Reference vertices:", len(reference_vertices))
print("Test vertices:", len(test_vertices))
print("Average correspondence distance:",
      np.mean(distances))

print("First 10 correspondences:")

for i in range(min(10, len(test_vertices))):
    print(
        "Test vertex",
        i,
        "-> Reference vertex",
```

```

indices[i, 0]
)

fig = plt.figure(figsize=(10, 5))

ax1 = fig.add_subplot(121, projection="3d")
ax1.scatter(
    test_vertices[:, 0],
    test_vertices[:, 1],
    test_vertices[:, 2],
    s=2
)
ax1.set_title("Test Face Mesh")

ax2 = fig.add_subplot(122, projection="3d")
ax2.scatter(
    corresponding_vertices[:, 0],
    corresponding_vertices[:, 1],
    corresponding_vertices[:, 2],
    s=2
)
ax2.set_title("Corresponding Reference Points")

```

OUTPUT

Graph-Based Learning for 3D Face Shape Correspondence

Reference Face Mesh



Test Face + Corresponding Landmarks

